Rajshahi University of Engineering & Technology

Department Of Mechatronics Engineering

AI-Enhanced Stereo Matching for High-Accuracy Depth Mapping and 3D Reconstruction

Course No: MTE 4200
Course Title: Project and Thesis

SUPERVISED BY
Md Zunaid Hossen
Lecturer
Department of Mechatronics Engineering

PRESENTED BY
Md Raihanul Haque Rahi - 2008011
Nazib Abrar - 2008026
Department of Mechatronics Engineering
Date: April 28, 2026

§ 00 AGENDA

Outline.

01 Introduction 03

02 Motivation 04

03 Problem Statement 05

04 Objective 06

05 Literature Review 07

06 Research Gap 08

07 Proposed Solution 09

08 System Overview 10

09 Architecture & Modules 11–16

10 Training & Loss 17–19

11 Results & Analysis 20–24

12 Discussion & Limitations 25

13 Impact & Contributions 26

14 Conclusion & Future Work 27

Stereo vision: Depth and Disparity

Two calibrated cameras see the same scene from slightly offset viewpoints. The pixel-level *shift* between them; the **disparity** d is inversely proportional to **depth**.

Close objects shift a lot between views: *large* disparity

Distant objects barely shift: *small* disparity

Depth recovery from disparity is a **purely geometric** post-processing step

FORMULA: DEPTH FROM DISPARITY

$$Z = \frac{f \cdot B}{\hat{d}}$$

Z metric depth (m)

f focal length (px)

B stereo baseline (m)

d predicted disparity (px)

Accuracy lives on servers. Deployment lives at the edge.

HEAVY NETWORKS

30–80 M params

PSMNet, RAFT-Stereo, CREStereo: desktop-GPU class; unsuitable for drones & mobile robots.

EDGE BUDGET

< 3 M params

Jetson Orin Nano class devices need compact, INT8-friendly models for real-time operation.

CLASSICAL METHODS

> 5.0 px EPE

SGM and block-matching fit on edge but collapse on textureless and reflective regions.

> *Robotics, ADAS, and low-power autonomy all need real-time depth, on hardware that can't run the state of the art.*

How do we recover dense, accurate depth under edge-scale compute, without giving up deep-learning accuracy?

Existing high-accuracy stereo networks are too heavy for on-device inference; existing lightweight pipelines lose unacceptable accuracy in textureless, reflective, or foliage-heavy regions.

03

Objective.

- To design a *computationally efficient* stereo matching pipeline that leverages *AI-based disparity refinement* to enhance depth estimation on **resource-limited platforms**.
- To design an architecture that can withstand **camera imperfections** in terms of rectification.

EFFICIENT

Parameters < 1 M · target < 40 ms/frame on Jetson Orin Nano.

AI-REFINED

Iterative deep refinement on top of a single coarse cost volume.

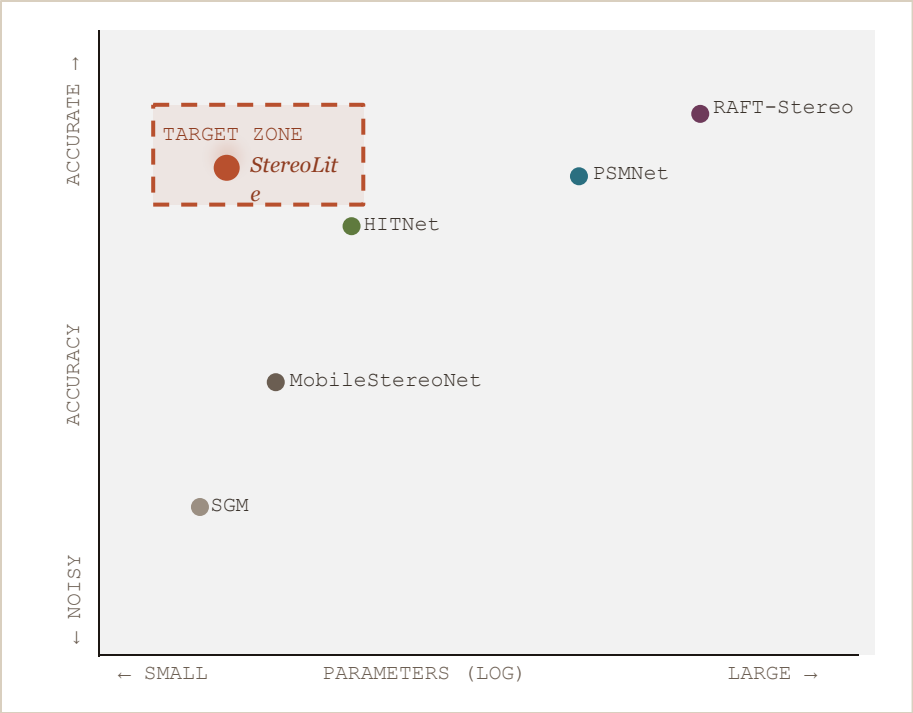
DEPLOYABLE

End-to-end trainable with multi-scale supervision and INT8 path.

Five ideas we build on.

METHOD	CORE IDEA	PARAMS	RELEVANCE TO STEREOLITE
GC-Net Kendall et al., 2017	First end-to-end 3D cost-volume regression	3.5 M	Established the cost-volume backbone.
PSMNet Chang & Chen, 2018	Pyramid pooling + stacked hourglass 3D CNN	5.2 M	Strong, but 3D conv cost unfit for edge.
HITNet Tankovich et al., 2021	Tile-hypothesis planes (d, s□, s□, f, c)	0.6 M	**Borrowed:** slanted tile state.
RAFT-Stereo Lipson et al., 2021	Recurrent residual refinement	11 M	**Borrowed:** iterative updates, convex upsample.
MobileStereoNet Shamsafar et al., 2022	MobileNet backbone for stereo	2.4 M	Edge-oriented, but weaker boundary accuracy.
StereoLite (Ours)	**Tile-hypothesis + RAFT refinement + convex up**	**0.87 M**	**Synthesis: one coarse CV + deep refinement.**

A gap in the accuracy × size plane.



Top-right cluster - highly accurate but >10 M params, desktop-GPU only.

Bottom-left cluster - deployable but poor on hard regions.

Under-populated: < 1 M params *and* competitive EPE.

StereoLite aims squarely for the small-&-accurate corner.

StereoLite at a glance.

TRAINABLE PARAMS

0.87 M

8.7 MB fp32 checkpoint; INT8 quantisation planned for Jetson.

LATENCY · RTX 3050

54 ms

Per 512×832 stereo pair; ~25–40 ms expected on Orin Nano INT8.

VAL. EPE

1.54 px

Scene Flow Driving · 200 held-out pairs.

IDEA · 01

Pretrained encoder. Truncated MobileNetV2 reuses ImageNet features at 4 scales.

IDEA · 02

One cost volume. Built only at 1/16 scale. cheap, but rich enough to init.

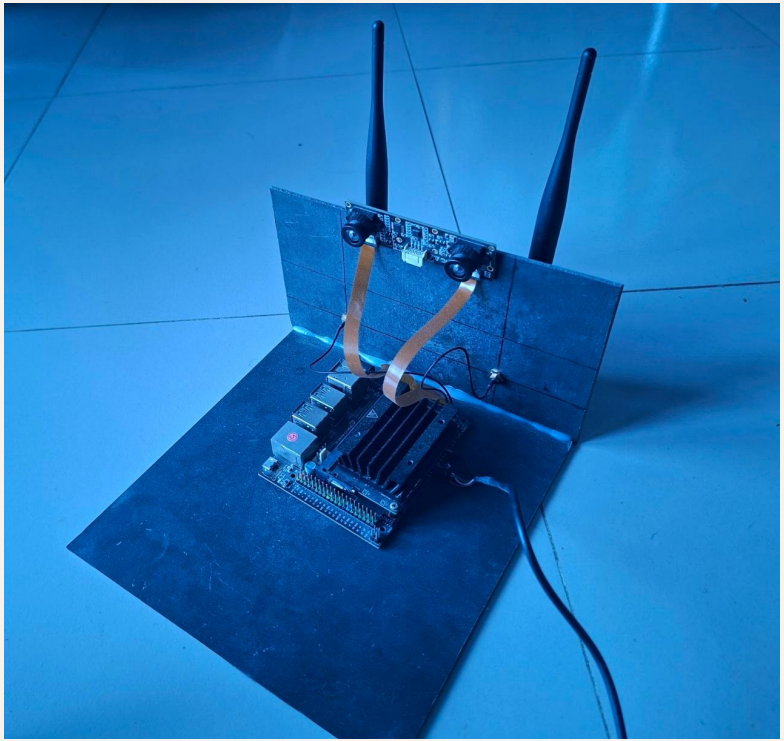
IDEA · 03

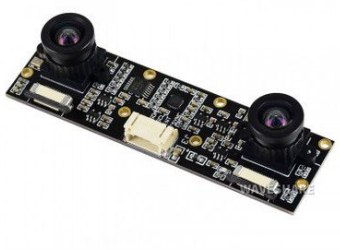
Iterative refinement. 2+3+3 residual updates over tile hypotheses.

IDEA · 04

Learned upsample. Convex 9-neighbour weights for boundary sharpness.

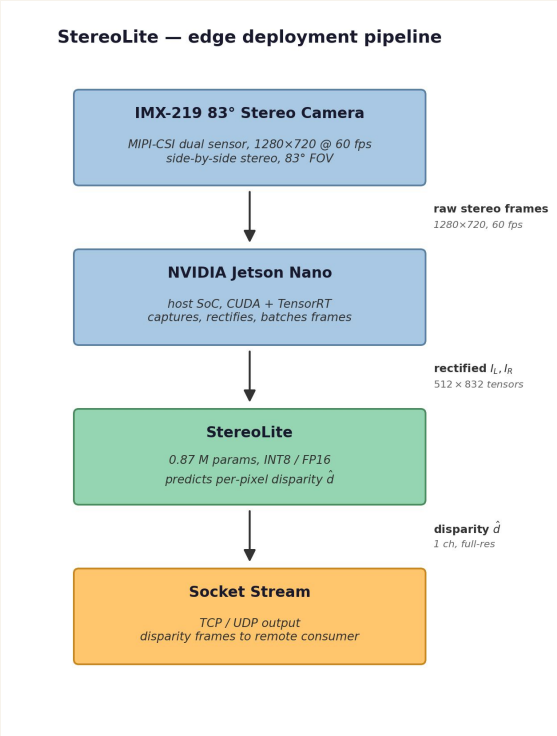
Development and Testing Setup: Hardware





IMX 219-83 Camera

NVIDIA Jetson Nano 4GB



Developed Model's(StereoLite) End-to-end architecture.

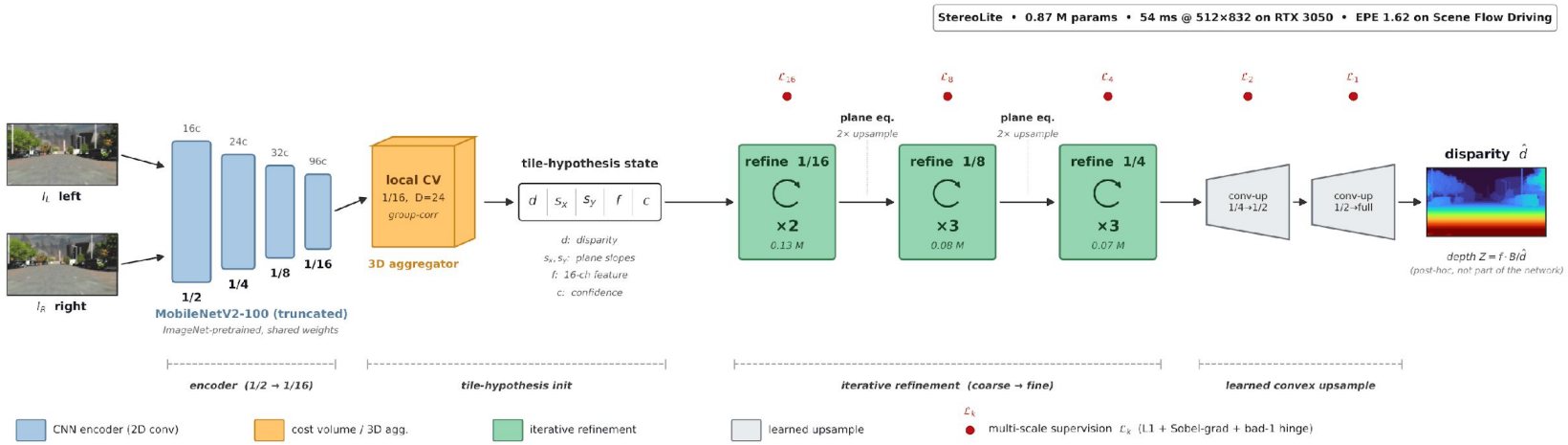


FIG. 1 – Data flows left → right. Red dots mark multi-scale supervision points · $L = \ell_1 + \lambda_g \ell_{grad} + \lambda_h \ell_{hinge}$.

Pretrained, truncated, shared.

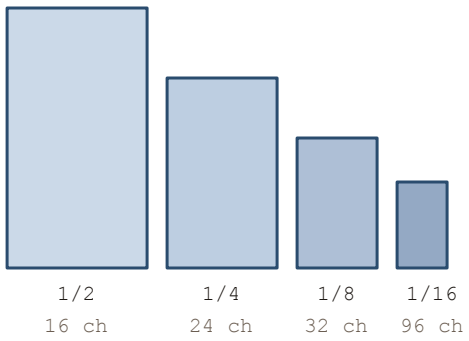
MobileNetV2-100 , weights from ImageNet: edges, corners, textures come from this layer.

Shared encoder on both left and right - features are directly comparable.

Truncated after stage four. Drops ~1 M deep-block params we never consume.

Input normalised inside the module; end-to-end fine-tuned.

FEATURE PYRAMID



Channels grow as spatial resolution shrinks - typical convolutional trade-off.

> *0.54 M of the 0.87 M total params live here - the rest is refinement.*

One cost volume. Only once.

Group-correlation CV at 1/16 only. 8 groups, 24 disparity candidates

3D aggregator, two Conv3D + GN + SiLU layers

Soft-argmin over disparity → initial d_0

Confidence c_0 = peak softmax probability

Slopes (s_x, s_Y) initialised to zero; feature f_0 from a single head on $f_{L,16}$

TILE STATE · T_K

d	Disparity value (px at 1/k units)
s□	Left-right slope of the surface
s□	Top-bottom slope
f	16-ch learned "memory" - refinement reads/writes
c	Confidence in [0, 1]

A *slanted little plane* per tile, not just a flat patch - captures roads & walls gracefully. **(HITNet)**

> *Cost volumes are expensive; we build one where it's cheapest . and use it to initialise, not regress.*

Small residuals, repeated.

Each `TileRefine` iteration performs:

Warp $f_{R,k}$ by current disparity via horizontal `grid sample`.

Concat $[f_{L,k}, \mathrm{warp}(f_{R,k}), f, d, s_{\square}, s_{\square}, c]$.

3-layer trunk (3×3, hidden 48, GN, SiLU).

Predict residuals Δd, Δs_x, Δs_y, Δf, Δc from five 1×1 heads.

Update: $d \leftarrow \mathrm{softplus}(d + \Delta d)$; $c \leftarrow \sigma(2c - 1 + \Delta c)$.

COARSE → FINE SCHEDULE

SCALE	ITERATIONS	PARAMS
1/16	×2	0.13 M
1/8	×3	0.08 M
1/4	×3	0.07 M
Total	**8 iters**	**0.28 M**

Why residuals? RAFT showed that many small corrections converge more accurately than one large regression. and each iteration is tiny, so compute stays low.

Plane-equation upsample.

Between refinement scales each parent tile spawns **four children** . Child disparity inherits from the parent's slopes:

$$d_{child} = 2 \cdot d_{parent} + (2 \cdot s_x) \cdot \Delta x + (2 \cdot s_y) \cdot \Delta y$$

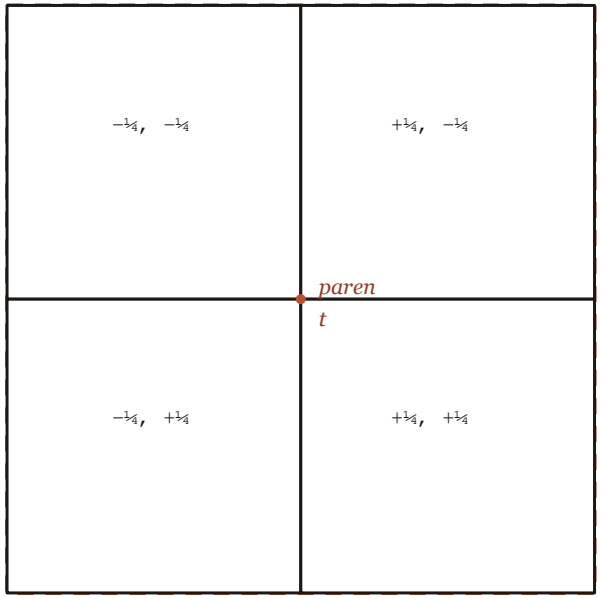
$\Delta x, \Delta y \in \{\pm 0.25\}$ - child offsets in parent units

Slopes, features, and confidence are **bilinearly upsampled**.

Keeps the cost volume at a **single scale** - no rebuild at each finer level.

Recovers **sub-pixel accuracy** through the learned slopes.

PARENT → 4 CHILDREN



Sharp boundaries without another CV.

RAFT-style convex upsampling. For every fine pixel the network predicts **nine weights that sum to one** , and the fine disparity is the convex average of the 9 coarse neighbours.

Mask conditioned on encoder features at the *destination* scale.

Learned weights attend to **image edges** → sharper than bilinear.

Two blocks chained: `1/4 → 1/2`, `1/2 → full`.

Avoids the compute of a third cost volume at fine resolution.

9-NEIGHBOUR WEIGHTED AVERAGE

$\hat{d} = \Sigma\, w_i \cdot d_i\,,\ \Sigma w_i = 1$

Multi-scale loss, three terms.

$$L = \sum_k w_k \left[\ell_1^{(k)} + \lambda_g \ell_{grad}^{(k)} + \lambda_h \ell_{hinge}^{(k)} \right] + \lambda_s \ell_{smooth}(\hat{d}, I_L)$$

ℓ_1 · L1 PIXEL ERROR

Average absolute pixel error on valid GT pixels. The workhorse term.

ℓ_{GRAD} · SOBEL GRADIENT

L1 on Sobel-(x, y) gradients of the disparity, keeps boundaries crisp.

ℓ_{HINGE} · BAD-1 HINGE

Extra penalty when error > 1 px, discourages large outliers.

Defaults · $\lambda_g = 0.5$ · $\lambda_h = 0.3$ · $\lambda_s = 0.02$ · $w_k = \{0.2, 1.0, 0.3, 0.5, 0.7, 1.0\}$ over {1/32, 1/16, 1/8, 1/4, 1/2, full}

Training recipe.

OPTIMISATION

Optimiser	AdamW (wd 1e-5)
LR Schedule	OneCycle
Peak LR	8×10^{-4}
Warmup	5%
Precision	fp16 + GradScaler
Epochs	30

DATA & HARDWARE

Dataset	Scene Flow Driving
Training pairs	4,200
Held-out val pairs	200
Input resolution	512 × 832
Effective batch	16 (8 × 2 GPU · DDP)
Hardware	Kaggle T4 × 2
Wall-clock	≈ 2 h 10 min

> *Augmentation: right-image random-erase (p = 0.3, 1–2 patches of 2–10% area) - forces robust matching.*

Every parameter, accounted for.

COMPONENT	PARAMS	SHARE
MobileNetV2-100 encoder (truncated)	0.54 M	62.0%
Tile init · CV + 3D agg. + feat head	0.03 M	3.4%
Refine 1/16 · ×2 iters	0.13 M	14.9%
Refine 1/8 · ×3 iters	0.08 M	9.2%
Refine 1/4 · ×3 iters	0.07 M	8.0%
Convex upsample · 1/4 → 1/2	0.01 M	1.1%
Convex upsample · 1/2 → full	0.01 M	1.1%
Total trainable	**0.874 M**	**100%**

SHARE OF 0.87 M

Encoder 62%

Refine 1/16 · 15%

Refine 1/8 · 9%

Refine 1/4 · 8%

Cost volume · 3%

Upsample · 2%

Checkpoint size (fp32) · **8.7 MB** · BN buffers · 0.032 M

Results.

30 epochs on Scene Flow Driving · 7,850 training steps · training curves, qualitative progression, and final-checkpoint metrics on 200 held-out validation pairs.

04

Loss drops two orders of magnitude.

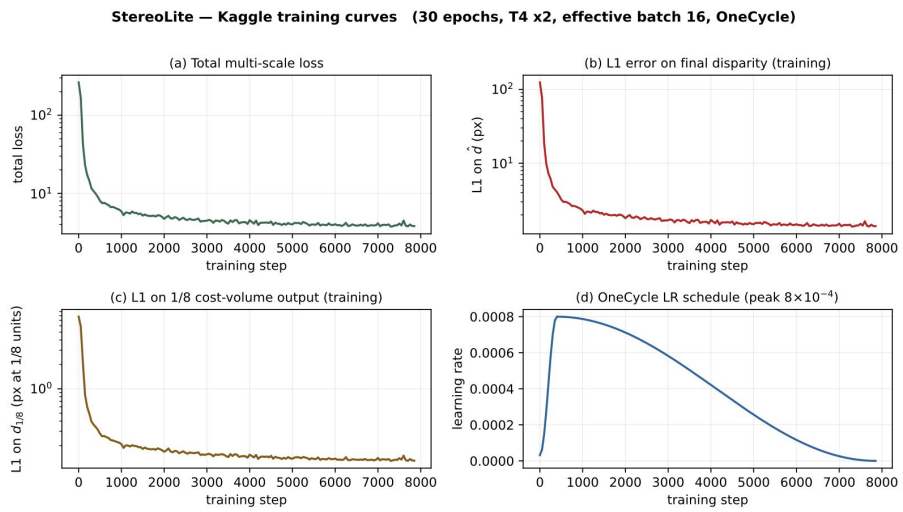


Figure 2: Kaggle training curves over 30 epochs. (a) total multi-scale loss (log scale); (b) L1 on the full-resolution disparity; (c) L1 on the 1/8 cost-volume output; (d) OneCycle learning-rate schedule.

FIG. 2 – 30 epochs · T4 × 2 · OneCycle LR peak 8×10^{-4}.

(A) TOTAL MULTI-SCALE LOSS

Drops monotonically by ~2 orders of magnitude.

(B) L1 ON FINAL DISPARITY

Converges to ~1.4 px without overfitting.

(C) L1 ON 1/8 CV HEAD

Plateaus at 0.13 px (≈1 px full-res) - a lower bound.

(D) ONECYCLE LR

5% warmup, cosine decay over the remaining 95%.

From global structure to boundary sharpness.

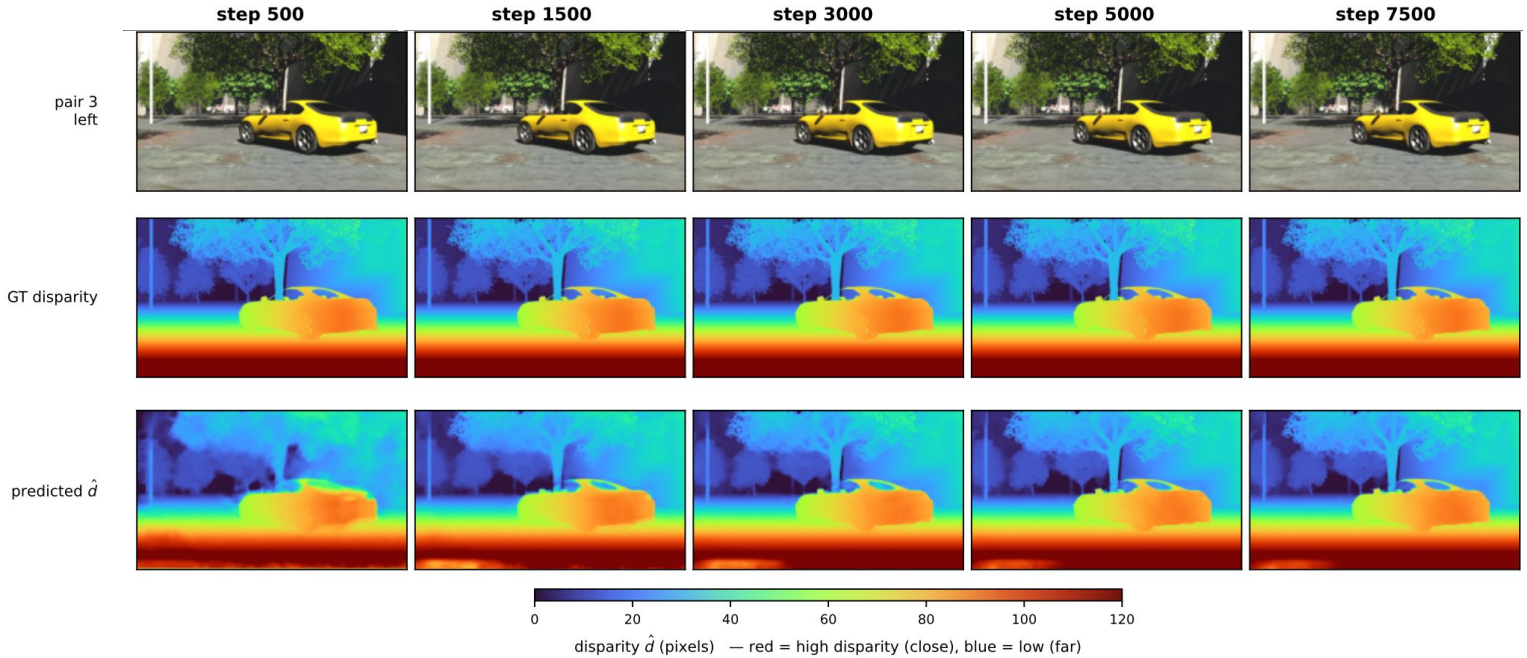


FIG. 3 – Two held-out pairs at checkpoints 500 → 7,500. Rows: left image · GT disparity · predicted $\hat{d}$. TURBO colourmap (red = close, blue = far).

Final metrics.

VALIDATION EPE

1.54 px

200 held-out pairs.

BAD-1

25.2 %

Dominated by sparse far-field GT.

TRAIN L1 (FINAL)

1.40 px

Small train↔val gap · clean generalisation.

CV HEAD L1 (1/8)

0.13 px

≈ 1 px at full resolution.

LATENCY · RTX 3050

54 ms

CHECKPOINT

8.7 MB

TRAINING STEPS

7,850

Reading the numbers honestly.

OBSERVATION 01 - CV HEAD PLATEAU

The 1/8-scale CV L1 plateaus near **0.13 px** (≈ 1 px at full resolution). This is an *effective lower bound* on final EPE until we deepen or multi-scale the cost volume.

OBSERVATION 02 - BAD-1 IS MISLEADING

A 25% bad-1 looks high, but is dominated by a handful of pairs with sparse far-field GT. **Mean EPE of 1.54 px** is the more representative number on Scene Flow Driving.

What the architecture earned:

Tile-hypothesis state captures slanted surfaces: roads, walls, reflective ground gracefully.

Iterative refinement converges cleanly without overfitting, visible in the narrow train↔val gap.

Convex upsample preserves boundaries that bilinear upsampling would smear.

What StereoLite does not do (yet).

LIMITED DATASET

Trained on Scene Flow Driving and custom generated dataset. For competing against existing models. It must be trained against other standard ones

NO REAL LIFE QUALITATIVE MEASUREMENT

The real life inference accuracy is measured against FoundationStereo. Access to 3D lidar would have provided accuracy on real life scenarios.

LIMITED STRESS TESTING ON RECTIFICATION

Inference on rectified image and non rectified image on current hardware shows marginal difference. More stress testing needed.

Confidence channel is **not absolutely calibrated.** used internally by the refinement stack, not as a trust score.

Bad-1 is sensitive to far-field GT sparsity; more robust metrics (e.g. D1-all) would be fairer.

Why this matters.

Contributions

Demonstration that **HITNet tile hypotheses + RAFT refinement** can be combined in < 1 M parameters without collapsing accuracy.

A **reproducible reference pipeline,** multi-scale supervision, training curves, public Scene Flow numbers.

A concrete **deployment path** to real-time stereo on Jetson-class edge hardware.

Where it helps

AUTONOMOUS DRONES

Obstacle avoidance where weight and wattage budgets are hard limits.

MOBILE ROBOTICS

SLAM front-end without a LiDAR - depth from two cheap cameras.

LOW-POWER ADAS

Forward-collision warning on embedded ECUs without desktop GPUs.

RESEARCH BASELINE

A small, honest baseline for the < 1 M-parameter stereo regime.

Conclusion.

StereoLite shows that a careful combination of a pretrained lightweight encoder, a single coarse cost volume, iterative tile refinement, and learned convex upsampling can meaningfully push the accuracy-versus-size Pareto front.

Future Work - the open items.

NEXT · 01

Full Scene Flow pretraining - Driving + Monkaa + FlyingThings3D (~35 k pairs).

NEXT · 02

KITTI 2015 fine-tune - report bad-3 on the public benchmark.

NEXT · 03

Cross-domain eval - Middlebury 2014 & ETH3D.

QUESTIONS & DISCUSSION

Thank you.

Code, logs, and training curves are available on request.

Contact · abarar@nazib@gmail.com, raihanulhaque007@gmail.com

Q&A